

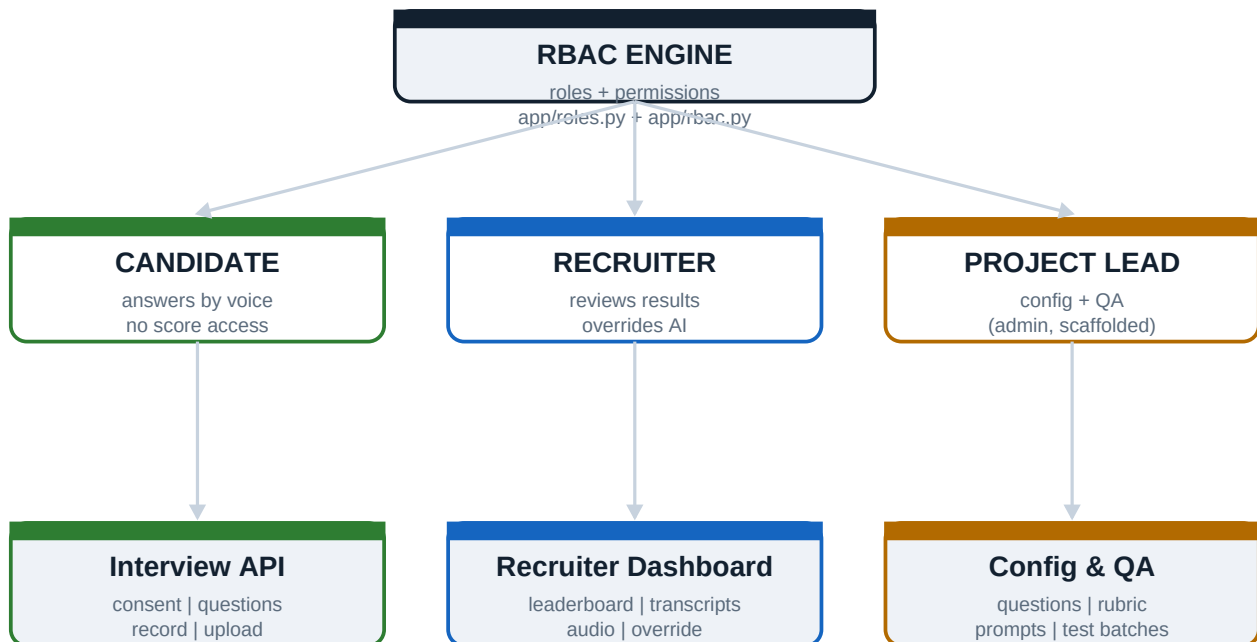
Backend RBAC

Automated Asynchronous AI Interviewer · Role-Based Access Control (based on SRS-2.3) · Lim Hoka (Backend)

1. Overview

Access to the backend is controlled by **role**. The three roles are the **SRS-2.3 user classes** — Candidate, Recruiter, and Project Lead (System Administrator) — and each role's permissions map directly to what the SRS says that class may do. Each request carries a signed session token that names the caller's role; FastAPI dependencies check that role (or a required permission) before the endpoint runs. A violation returns **403** in the standard error envelope. Roles and permissions live in one place (`app/roles.py`) and are enforced by guards in `app/rbac.py`.

2. Architecture



RBAC engine grants each role a set of permissions, which gates the resources that role can reach.

3. Roles

Role	SRS user class (SRS-2.3)	Sign-in
candidate	External interviewee — no scoring visibility	Email magic-link

recruiter	Trusted reviewer — leaderboard/override	Email magic-link
admin	Project Lead / System Administrator	Scaffolded (not issued yet)

4. Permission matrix

Permission	candidate	recruiter	admin
give_consent	✓	—	—
answer_interview	✓	—	—
upload_response	✓	—	—
view_leaderboard	—	✓	✓
view_candidate	—	✓	✓
play_audio	—	✓	✓
override_score	—	✓	✓
view_budget	—	✓	✓
manage_questions	—	—	✓
manage_rubric	—	—	✓
manage_prompts	—	—	✓
run_qa	—	—	✓

✓ = granted · — = denied. Admin inherits every permission.

5. Endpoint protection

Endpoint	Role · permission	Built?
POST /auth/magic-link, /auth/verify	public	yes
GET /auth/me	any session	yes
POST /interview/consent	candidate · give_consent	yes
GET /interview/questions	candidate · answer_interview	yes
POST /interview/responses	candidate · upload_response	yes
GET /interview/responses/{id}	candidate	yes
POST /interview/events/tab-out	candidate	yes
GET /interview/status	candidate	yes
GET /jobs/{id}/leaderboard	recruiter · view_leaderboard	planned
GET /candidates/{id}	recruiter · view_candidate	planned
GET /responses/{id}/audio	recruiter · play_audio	planned
GET /system/budget-status	recruiter · view_budget	planned

Questions / rubric / prompts config	admin · manage_*	planned
-------------------------------------	------------------	---------

6. How it is enforced

```
from fastapi import Depends
from app.rbac import require_role, require_permission
from app.roles import Role, Permission
```

```
# by role
@router.get("/jobs",
    dependencies=[Depends(require_role(Role.RECRUITER, Role.ADMIN))])
```

```
# by permission
@router.post("/jobs",
    dependencies=[Depends(require_permission(Permission.MANAGE_JOBS))])
```

```
# a wrong role/permission -> 403 { "error": { "code": "FORBIDDEN", ... } }
```

Note: the admin role is defined and enforced, but not issued yet — the frozen API Contract v1 only mints candidate/recruiter tokens. Enabling admin sign-in (for Project-Lead job/rubric management) is a small contract-v1.1 step.